第5回 演習・宿題：ツールを MCP サーバーにして使う

この回のゴール

- 第3回で作ったツールを **MCP サーバー**として切り出し、同じエージェントから **MCP 経由**で使える
- モデルに見えている「ツールの名前・説明文・引数」を読める
- **設定ファイルを書き換えるだけ**で、既製の外部ツールを足せる

準備

前の回までの作業が終わっている場合は、`main` から `lec05` ブランチを作ります。この回から依存パッケージが増えているので、`uv sync --frozen` を一度実行してください。

```
git switch main && git switch -c lec05
uv sync --frozen
```

終わっていない場合は、講座 repo の開始時点のコード（`lec05-start` タグ）からブランチを作れます。

```
git remote -v                       # upstream がなければ次の行で登録する
git remote add upstream https://github.com/matsuolab/lecture-ai-engineering-2026.git
git fetch upstream --tags
git switch -c lec05 lec05-start
uv sync --frozen
```

演習（30分）

1. モデルに見えているツールを一覧する

```
uv run python -m app.agent --list-tools          # 第3回：関数を直接登録したツール
uv run python -m app.agent --mcp --list-tools    # 第5回：MCP サーバーから受け取ったツール
```

どちらも LLM は呼ばないので、API キーなしで動きます。2つ目は `app/mcp_servers.json` に書かれたサーバー（`app/mcp_server.py`）をサブプロセスとして起動し、公開されているツールの名前・説明文・引数を表示します。

- 名前が `glossary_search_glossary` のように「**サーバー名_**」**付き**になっていることを確認しましょう。複数のサーバーに同じ名前のツールがあっても区別できるようにするためです
- 説明文は `app/tools.py` の docstring そのものです。**モデルから見れば、関数ツールも MCP のツールも同じ形**をしています

2. エージェントから MCP 経由で使う

```
uv run python -m app.agent --mcp --trace "AIE2026の講座の宿題にある「3レベル」とは何ですか？"
```

第3回（`--mcp` なし）と比べて、回答・ツール名・`requests` の回数がどう違うかを見ます。内部では、`app/agent.py` の `build_mcp_agent()` が設定ファイルを読み、サーバーを起動して、ツールをエージェントに渡しています。

3. ツールを1つ公開する

`app/mcp_server.py` の `TODO(第5回 演習)` の場所で、`get_lecture_summary` も MCP で公開します（`search_glossary` と同じ書き方で 1 行）。

```
uv run python -m app.agent --mcp --list-tools
uv run python -m app.agent --mcp --trace "第2回では何を学びますか？"
uv run pytest -q
```

`--list-tools` に `glossary_get_lecture_summary` が増え、`--trace` でそれが呼ばれることを確認します。第3回で `make_quiz` を作った人は、それも同じ方法で公開できます。

4. 既製の外部ツールを、設定だけで足す

`app/mcp_servers.json` に、公開されている **fetch サーバー**（URL のページを取得して Markdown で返す）を足します。コードは書きません。

```
{
  "mcpServers": {
    "glossary": {
      "command": "uv",
      "args": ["run", "python", "-m", "app.mcp_server"]
    },
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"]
    }
  }
}
```

```
uv run python -m app.agent --mcp --list-tools
uv run python -m app.agent --mcp --trace "https://modelcontextprotocol.io/ の冒頭を日本語で
```

```
3行に要約して"
```

- 初回は `uvx` がサーバーをダウンロードするので、起動に 10 秒ほどかかります
- `--trace` に `fetch_fetch({'url': ...})` が出れば、**モデルが選んだ URL に Codespace から実際にアクセス**しています。どこへ何を送るツールかを把握したうえで足すのが、外部ツールを使うときの基本です
- `INSTRUCTIONS` は用語集を前提に書いてあります。URL の質問で fetch が呼ばれないときは、「URL を渡されたら fetch で内容を読む」のような 1 行を `INSTRUCTIONS` に足してみましょう

5. 提出

`lectures/05/answer.md` を書き、`lec05` → 自分の fork の `main` へ PR を出します。

宿題（どれか1つで完了）

レベル	内容
基礎	サーバーを `uv run python -m app.mcp_server --http` で起動し、設定ファイルの `glossary` を `{"url": "http://127.0.0.1:8000/mcp"}` に替えて同じ質問をする。stdio と Streamable HTTP で、起動の仕方・プロセス・ポートがどう違うかを記録する。あわせて、ツールの docstring を書き換えると `--list-tools` と呼ばれ方がどう変わるかを比べる
実装	第3回の `make_quiz`（なければ新しいツール）を MCP サーバーに追加し、TestModel を使ったテストを `tests/test_mcp_server.py` に足す。引数の範囲チェック（例：存在しない回なら分かりやすいメッセージを返す）を入れ、MCP 越しのエラーがモデルにどう見えるかを `--trace` で確かめる
発展	同じ用語集サーバーを **Copilot Chat**（VS Code の MCP クライアント）から使う（下の手順）。「サーバーを1つ作れば、クライアントを選ばずに使える」ことを確かめ、エージェント（`app/agent.py`）からの使い方と比べる。ツールが呼ばれるまでの流れ（誰がサーバーを起動するか、呼び出し前に人の承認があるか、結果がどう見えるか）の違いを記録する

発展の進め方

Copilot Chat は VS Code の MCP クライアントです。リポジトリに `.vscode/mcp.json` を置くと、Chat の **Agent** モードでサーバーのツールが使えます（形式は `app/mcp_servers.json` の `mcpServers` ではなく `servers`。コマンドは作業フォルダ＝リポジトリのルートで実行されます）。

```
{
  "servers": {
    "aie-glossary": {
      "type": "stdio",
      "command": "uv",
      "args": ["run", "python", "-m", "app.mcp_server"]
```

```
    }
  }
}
```

1. Codespaces の VS Code で上のファイルを作る。`servers` の上に **Start** が出たら押す（出ないときはコマンドパレットで **MCP: List Servers** → `aie-glossary` → Start）。サーバーは Codespace の中で起動します
2. Copilot Chat を開き、モードを **Agent** にする。ツール（🔧）の一覧に `aie-glossary` の `search_glossary`（演習 3 を終えていれば `get_lecture_summary` も）が出るか確認する
3. 「AIE2026 の宿題の 3 レベルとは？」と聞く。ツールの実行前に確認（**Continue**）を求められたら許可し、用語集のツールが呼ばれて回答に使われるかを見る
4. `answer.md` に、Chat に表示されたツール呼び出しと、`uv run python -m app.agent --mcp --trace` の表示を並べて書く
5. `.vscode/mcp.json` は PR に含めてよい（API キーのような秘密は書かないこと）
6. Copilot Free でも Agent モードと MCP は使えます。ただしチャットの回数には月の上限があるので、試す回数は数回にとどめましょう
7. 会社や学校の組織が管理する Copilot（Business / Enterprise）では、組織のポリシーで MCP が無効になっていることがあります。個人アカウントで試してください

注意

- MCP サーバーは**コードを実行するプログラム**です。出所の分からないサーバーを設定ファイルに書かないでください
- fetch サーバーは、モデルが決めた URL にアクセスします。社内の URL や個人情報を含む URL を渡さないでください
- ツールが増えると、モデルに渡す説明文が増え、1回あたりのトークン数と呼び分けの難しさが増します。`--list-tools` で何個のツールを渡しているかを意識しましょう

参考

- MCP の公式サイト: https://modelcontextprotocol.io/
- Pydantic AI の MCP クライアント: https://pydantic.dev/docs/ai/mcp/client/
- MCP の Python SDK（サーバーの書き方）: https://github.com/modelcontextprotocol/python-sdk
- 公開されている MCP サーバー: https://github.com/modelcontextprotocol/servers
- VS Code で MCP サーバーを使う: https://code.visualstudio.com/docs/copilot/customization/mcp-servers